

SubSwipe

Condensed Product Requirements Document - 20-page engineering edition

Purpose: Give an engineer or coding AI enough product, UX, logic, data, and acceptance detail to build a working React Native MVP without losing the core product differentiators.

Product thesis

SubSwipe is a two-sided student sublease marketplace that behaves like a matching app first and a housing classifieds product second. Renters and subletters receive ranked matches, swipe left or right, and connect only when there is mutual interest.

The product is designed around a simple economic problem: every empty day in a lease is value that cannot be recovered later. SubSwipe therefore combines housing search with pricing, date, and demand intelligence.

Non-negotiable MVP systems

- **Urgency Pricing:** recommend price reductions as vacancy approaches and quantify the expected change in match pool.
- **Date Match Score + two-sided swiping:** compare exact date overlap and let both renters and sellers swipe.
- **Reverse Matching:** automatically surface compatible renters to sellers even before the renter discovers the listing.
- **Partial-Fill Matching:** combine multiple renters across non-overlapping date windows to cover one vacancy.
- **Demand Heatmap / Demand Intelligence:** show what renters are actively searching for and feed that data into pricing and matching.

Recommended stack

Layer	Recommendation
Frontend	React Native + Expo + TypeScript
Navigation	Expo Router
Gestures / animation	React Native Gesture Handler + Reanimated
Backend	Supabase + PostgreSQL
Storage	Supabase Storage
Auth	Supabase Auth

02 Problem, Users, and Product Positioning

Problem

Students often leave a college town for summer, internships, co-ops, graduation, or temporary work while still owing rent. At the same time, other students need short-term housing for a few weeks or months. Current solutions are fragmented across Facebook groups, GroupMe, Reddit, spreadsheets, word of mouth, and generic housing sites.

Primary users

- **Renter:** needs short-term housing with specific dates, budget, location, and preferences.
- **Seller / subletter:** is already responsible for a lease and wants to fill some or all of the remaining period.
- A single account can use both modes; mode switching should not require a second account.

Positioning

Do not position SubSwipe as "another student sublease board."

Position it as a **two-sided matching and rent-recovery marketplace** that uses exact-date matching, urgency pricing, reverse matching, demand intelligence, and multi-renter combinations.

Core renter value: find housing that actually matches your dates and constraints.

Core seller value: find the best renters faster and recover more rent that would otherwise be lost.

Success definition for the MVP

- A renter can create preferences and swipe through ranked sublease cards.
- A seller can create a listing and immediately see compatible renters.
- A mutual right swipe creates a match.
- Seller can see rent-at-risk, demand, pricing recommendations, and partial-fill options.
- The five core systems visibly work together in one demo flow.

03 Core System Overview

All five systems below are required. If one is missing, the core SubSwipe product is incomplete.

System	User-facing output	Primary data inputs
Urgency Pricing	Suggested price + predicted match-pool change	Vacancy timing, asking price, demand, budgets, competing listings
Date Match + Swiping	Date score, overall score, swipe deck, mutual match	Renter/listing dates, flexibility, budget, location, preferences
Reverse Matching	Ranked renter feed for seller	Active renter profiles + seller listing
Partial Fill	Multi-renter combination + coverage + recovered renter	Renter date windows + listing vacancy window
Demand Heatmap	Demand by date, budget, location, preferences	All active renter searches/profiles

System relationship

Demand should not be a decorative chart. It should power the rest of the product.

Demand Heatmap -> Urgency Pricing -> Reverse Matching -> Date Match -> Partial-Fill Matching

- Demand identifies what renters currently want.
- Urgency Pricing uses that demand to recommend a more competitive price.
- Reverse Matching recalculates how many renters become compatible.
- Date Match identifies the strongest individual matches.
- Partial-Fill logic searches for combinations when a single renter does not cover the full vacancy.

Engineering rule

Implement each system as reusable logic exposed to multiple screens. Do not hard-code values only for the seller dashboard or demo UI. Seeded data is acceptable, but the score and recommendation functions should actually compute from that data.

04 Core Feature 1 - Urgency Pricing

Goal

Help sellers minimize lost rent by recommending more competitive asking prices as their vacancy date approaches.

Required inputs

- seller asking price
- seller monthly obligation
- available start date
- current date / days until vacancy
- seller urgency level
- number of matching renters
- budget distribution of matching renters
- competing listing prices
- market demand from active renter profiles

Required output

Example recommendation

9 days until vacancy | Current: \$1,050/mo | Suggested: \$925/mo | Current pool: 17 renters | Predicted pool: 24 renters (+41%)

- Always show the reason for the recommendation.
- Show the predicted impact on the match pool.
- Do not automatically change the listing price.
- Seller must explicitly choose **Apply Suggested Price** or **Keep Current Price**.
- If price changes, update renter cards with a **PRICE DROP** badge where appropriate.

Urgency levels

Level	Meaning
Normal	Seller can wait for the right renter.
Need It Filled	Seller wants to find someone soon.
Urgent	Seller is leaving soon and prioritizes reducing vacancy loss.

MVP pricing logic

If days-to-vacancy falls AND match_count is low, recommend a lower price. Use median renter budget and competing prices to choose the recommendation. For the hackathon, a deterministic rule-based function is enough; no ML is required.

05 Core Feature 2 - Date Match Score and Two-Sided Swiping

Date Match Score

Compare exact date windows instead of broad terms like "summer housing." Date compatibility is the highest-weight factor in overall matching.

Example

Renter: June 3 - August 9 | Listing: May 28 - August 12 -> 98% Date Match

Date flexibility

- Exact dates
- +/- 3 days
- +/- 1 week
- Flexible

Overall Match Score

Factor	Weight
Date compatibility	40%
Price compatibility	25%
Location compatibility	20%
Housing preferences	15%

$\text{overall_match} = \text{date} * 0.40 + \text{price} * 0.25 + \text{location} * 0.20 + \text{preferences} * 0.15$

Renter swipe deck

- Swipe right = Interested
- Swipe left = Pass
- Tap card = open full listing
- X button = Pass
- Heart button = Interested
- Cards ordered from highest to lowest match score

Seller swipe deck

- Same gesture system, but cards represent renter profiles.
- Seller right swipe indicates willingness to rent to that student.
- A mutual right swipe creates a match and shows an animated "It's a Match" screen.

React Native requirements

Use Gesture Handler + Reanimated. Card follows finger, rotates slightly, shows LIKE/PASS overlay, exits screen, then reveals the next card. Persist every swipe to the backend.

06 Core Feature 3 - Reverse Matching

Definition

The seller should not wait for renters to discover the listing. After listing creation, SubSwipe automatically compares the listing against all active renter profiles and creates a ranked renter feed.

Example output

"12 verified students are currently looking for a place like yours."

Ranking factors

- date compatibility
- maximum renter budget vs asking price
- desired location / distance
- furnished preference
- parking preference
- roommate preference
- housing type
- overall match score

Important distinction

Reverse Matching must include renters who have not discovered or liked the listing yet. It is not the same as "people who viewed my listing."

Seller feed behavior

- Display renter cards highest score first.
- Show renter name, university, desired dates, budget, Date Match, overall Match, verification badge.
- Seller can swipe left/right or tap for profile details.
- When seller likes someone who already liked the listing, create a match immediately.

Backend flow

Seller publishes listing -> query active renter profiles -> calculate scores -> filter low-quality matches -> sort descending -> return renter deck -> store seller swipes.

07 Core Feature 4 - Partial-Fill Matching

Goal

Allow one vacancy period to be covered by multiple renters with different date windows.

Example

Listing: May 20 - August 20 Renter A: May 25 - June 30 Renter B: July 1 - August 18 Combined coverage: 95%
Estimated recovered rent: \$2,550

Required logic

- Find renters whose dates overlap the listing.
- Calculate each renter's individual coverage.
- Generate 2-renter combinations first; 3-renter combinations are optional for MVP.
- Reject combinations with significant date overlap or invalid ordering.
- Calculate uncovered days, coverage percentage, and estimated recovered rent.
- Rank combinations by coverage, estimated recovery, and match quality.

Seller presentation

Use a **Better Together** card and a timeline visualization showing the listing window and each renter's period.

Do not make this only a visual mock. The backend must compute the combination from renter date data.

MVP acceptance

- At least one seeded listing must produce a real 2-renter partial-fill recommendation.
- Coverage percentage must be computed from dates.
- Estimated recovered rent must be computed from asking price and covered days.

08 Core Feature 5 - Demand Heatmap and Demand Intelligence

Goal

Show sellers what active renters are searching for and use those insights to drive pricing and listing decisions.

Demand dimensions

- move-in / move-out periods
- price ranges
- city / university / area
- maximum distance or commute
- furnished preference
- parking preference
- housing type
- roommate preference

Example seller insight

Current demand near OSU June-August: HIGH Under \$900: VERY HIGH \$900-\$1,100: MEDIUM Furnished: 74% prefer it Parking: 48% require/prefer it Within 1 mile: 83% prefer it

Required behavior

- Heatmap data comes from active renter profiles/search preferences.
- Seller can see where their listing sits relative to demand.
- Pricing recommendations should use the demand data.
- Seller can see how a hypothetical change affects the number of compatible renters.
- A map visualization is optional: demand intelligence can be implemented with cards/bars for MVP

Do not treat the heatmap as a future enhancement. It is required in the MVP seller experience.

09 Renter Workflow

1. Authentication

- Sign up / log in with Supabase Auth.
- Collect name, email, university, password.
- For MVP, .edu email can display a verified-student badge after email verification.

2. Choose mode

- Find Housing
- List My Place
- User may switch later without creating a new account.

3. Renter onboarding

Step	Fields
Location	City, university, optional work/internship location
Dates	Move-in, move-out, date flexibility
Budget	Maximum monthly budget
Preferences	Furnished, parking, housing type, roommates, max distance/commute

4. Discover

- Load ranked listing cards.
- Show overall Match %, Date Match %, price, dates, distance, key amenities, urgency / price-drop badges.
- Swipe right/left or tap for details.
- Allow separate bookmark for Save; saving is not the same as liking.

5. Listing detail

- Photo carousel
- Match breakdown + explanation
- True monthly cost
- Available dates
- Amenities and roommates
- Seller identity + verification
- Pass / Interested actions

6. Match and negotiate

- Mutual interest opens Match Success screen.
- User can message, make offer, or continue swiping.
- Offers include monthly price and requested date window.

10 Seller Workflow

1. Create listing

Step	Fields
Property	Address, city, state, ZIP, university
Details	Room/apartment, bedrooms, bathrooms, furnished, parking, roommates, amenities
Photos	Camera or photo library; Supabase Storage
Availability	Available from / until
Costs	Current rent, utilities, parking, required fees
Asking price	Desired monthly sublease price
Urgency	Normal / Need It Filled / Urgent

2. Immediate financial context

- Calculate Rent at Risk from remaining lease period and obligation.
- Calculate approximate Daily Vacancy Loss = monthly obligation / 30.
- Display both before seller enters the discovery experience.

3. Reverse matching

- Immediately show count of compatible renters.
- Open ranked renter swipe deck.
- Seller can like/pass renters before any renter contacts them.

4. Seller dashboard

- Rent at Risk
- Daily Vacancy Loss
- Current asking price
- Suggested price
- Potential renters
- Mutual matches
- Rent Recovery Score
- Demand Heatmap
- Partial-fill opportunities
- Listing optimization recommendations

11 Mutual Match, Messaging, Offers, and Flexible Dates

Mutual Match

Condition: renter right-swipes listing AND seller right-swipes renter. Backend creates one match record. Frontend shows animated Match Success screen.

Match actions

- Message
- Make Offer
- View listing / renter profile
- Keep Swiping

Messaging

- One conversation per match.
- MVP supports text, timestamp, sender, read/unread state.
- Real-time updates via Supabase Realtime are recommended but not mandatory if polling is simpler.

Offer system

Renter submits monthly price + requested start/end dates. Seller can Accept, Reject, or Counter. Counteroffer produces a new proposed price/date state on the same negotiation thread.

Flexible-date negotiation

If there is a small date mismatch, highlight it rather than dropping the match. Example: listing begins June 1, renter needs June 5 -> show "4-day gap" and estimated financial value of those days, then offer **Ask About Flexible Dates**.

Secondary status states

Entity	Suggested states
Match	active, archived, closed
Offer	pending, accepted, rejected, countered
Listing	draft, active, matched, closed
Conversation	active, archived

12 Seller Intelligence: Rent Recovery Score and Listing Optimization

Rent Recovery Score

A 0-100 score representing how well positioned the listing is to recover the seller's remaining rent. This does not need predictive ML for the MVP.

Possible inputs: match count, date compatibility, price competitiveness, demand, flexibility, listing completeness, urgency.

Example

Current score: 54% Lower rent to \$900 -> 78% Allow partial renters -> 91% Allow +/- 5 day flexibility -> 94%

Make My Listing Work

If demand is weak, show actionable changes and re-run the compatibility query against active renters.

Current matches: 4 Reduce rent by \$75 -> 19 matches Allow earlier move-out -> 27 matches Add furniture -> 34 matches

True Cost

For renters, compare true monthly cost rather than base rent only: Rent + utilities + parking + required fees.

Activity indicators

- students interested
- offers pending
- high demand this week
- price dropped
- new renters matching listing

Seeded counts are acceptable for demo-only activity labels, but match counts and recommendation counts should come from actual seeded profile queries whenever possible.

13 Matching and Scoring Logic

Date score

Compute the percentage of the renter's requested date range covered by the listing, adjusted for stated flexibility. Perfect containment should score near 100. Partial overlap scores proportionally lower. No overlap = 0.

Price score

If asking price $\leq$ renter budget, score high. If asking price exceeds budget, decay score based on percentage over budget. Keep the function smooth so a \$20 difference does not collapse the score.

Location score

For MVP, use stored distance in miles or a simple geocoded distance if a mapping service is available. Internship mode may use commute distance/time when present.

Preference score

Evaluate furnished, parking, housing type, roommate preference, and other must-have/preferred flags. Required preference mismatches should penalize more than soft preferences.

Overall ordering

Filter impossible matches first (for example zero date overlap), then calculate overall score, then sort descending. Remove entities the user already passed on.

Match explanation

Every high score should be explainable to the user. Example: "Dates nearly perfectly align; \$50 under budget; furnished; parking included; 0.4 miles from campus."

14 Backend and Data Model

Table	Key fields
users	id, name, email, university, photo, verified, created_at
renter_profiles	user_id, city, university, work_location, dates, flexibility, max_budget, max_distance, preferences, active
listings	seller_id, address, location, dates, monthly_rent, asking_price, utilities, parking, rooms, furnished, urgency, status
listing_photos	listing_id, storage_url, position
swipes	user_id, target_type, target_id, direction, created_at
match_scores	renter_id, listing_id, overall, date, price, location, preference
matches	renter_id, seller_id, listing_id, status, created_at
offers	match_id, created_by, monthly_price, start_date, end_date, status
messages	match_id, sender_id, message, created_at, read_at
demand_metrics (optional cache)	university/city, period, budget bucket, renter_count, preference percentages
partial_fill_groups (optional)	listing_id, coverage_percentage, estimated_revenue
partial_fill_members (optional)	group_id, renter_id, start_date, end_date

Backend recommendation

Use Supabase/PostgreSQL. Store raw profile/listing data and compute scoring in database functions, Edge Functions, or application services. Favor readable deterministic logic over premature optimization.

15 React Native Architecture and Screen Map

Required frontend stack

- React Native
- Expo
- TypeScript
- Expo Router
- React Native Gesture Handler
- React Native Reanimated

Suggested app routes

```
/auth/login, /auth/signup, /auth/verify /onboarding/choose-mode
/onboarding/renter/{location,dates,budget,preferences}
/onboarding/seller/{property,details,dates,pricing,urgency}
/(renter-tabs)/{discover,matches,saved,profile} /(seller-tabs)/{renters,dashboard,matches,listing}
/listing/[id] /renter/[id] /match/[id] /chat/[id] /offer/[id] /partial-fill/[id] /demand
```

Primary screens

- Welcome / auth
- Renter onboarding
- Seller onboarding
- Renter Discover swipe deck
- Seller Renters swipe deck
- Listing detail
- Renter profile
- Match Success
- Matches
- Chat
- Offer
- Seller Dashboard
- Partial Fill
- Demand

Navigation

Renter tabs	Seller tabs
Discover	Renters
Matches	Dashboard
Saved	Matches
Profile	Listing

16 Reusable Components and Frontend Data Flow

Recommended components

SwipeDeck.tsx, ListingCard.tsx, RenterCard.tsx, MatchScore.tsx, MatchBreakdown.tsx, DateMatchBadge.tsx, UrgencyBadge.tsx, PriceDropBadge.tsx, TrueCostCard.tsx, RentAtRiskCard.tsx, RentRecoveryScore.tsx, SuggestedPriceCard.tsx, DemandHeatmap.tsx, DemandInsightCard.tsx, PartialFillCard.tsx, OfferCard.tsx, VerifiedBadge.tsx, EmptyState.tsx

Renter recommendation flow

Open Discover -> load active listings -> remove previously passed listings -> calculate scores -> sort -> return top cards -> render SwipeDeck.

Seller recommendation flow

Open Renters -> load active renter profiles -> calculate compatibility to seller listing -> remove passed renters -> sort -> render renter SwipeDeck.

Swipe flow

Right swipe -> animate -> persist swipe -> check opposite-side interest -> if mutual create match and open Match Success; otherwise load next card. Left swipe -> persist pass -> animate away -> load next card.

State handling

- Optimistically animate swipes, but roll back if persistence fails.
- Cache a small stack of upcoming cards for smooth interaction.
- Never resurface passed cards unless user resets preferences or history.
- After profile/listing changes, invalidate recommendation queries and recalculate.

17 Core Algorithms and Business Rules

Urgency Pricing - MVP rule

Use `days_to_vacancy`, `current_match_count`, `urgency_level`, median matching-renter budget, and competing prices. As vacancy approaches and match count remains low, lower the recommendation toward the renter-demand median. Return `suggested_price`, `current_pool`, `predicted_pool`, `percent_change`, `explanation`.

Demand calculation

Aggregate active renter profiles into date buckets, price buckets, location/distance buckets, and preference percentages. Cache if useful, but MVP can compute from seeded data on demand.

Partial-fill search

For each listing, collect partially overlapping renters. Evaluate pairs, sort each pair by start date, reject excessive overlap, compute union of covered days inside the listing window, and return top combinations by coverage and recovered rent.

Rent at Risk

Approximate remaining obligation across available dates using $\text{monthly obligation} / 30 * \text{vacancy_days}$. Keep calculation transparent and label as estimate.

True Cost

```
true_monthly_cost = asking_price + utilities_cost + parking_cost + required_fees
```

No legal automation in MVP

SubSwipe helps parties discover and negotiate. It does not claim landlord approval, execute legal lease transfers, verify lease permissibility, or guarantee sublease legality. Those are later integrations / workflows.

18 MVP Build Order and Demo Requirements

Build order

Phase	Deliverable
1	Expo + TypeScript + routing + auth
2	Renter and seller onboarding + database models
3	Match scoring + ranked recommendation queries
4	Two-sided SwipeDeck + persisted swipes
5	Mutual match creation + Match Success screen
6	Reverse Matching seller feed
7	Seller dashboard + Rent at Risk + Rent Recovery Score
8	Demand Heatmap + Urgency Pricing
9	Partial-Fill matching + visualization
10	Polish, seeded data, offers/chat if time

Required seeded data

- 10-15 renters
- 10-15 listings
- vary dates, budgets, location, furnished, parking, urgency
- seed data intentionally so all five core systems produce visible results

Required demo sequence

Renter swipes -> likes 96% match -> switch to seller -> show reverse matching -> seller likes renter -> It's a Match -> dashboard shows demand + urgency price recommendation -> partial-fill suggestion shows two renters covering ~95%.

19 Business Model, Future Roadmap, and Out-of-Scope Items

MVP monetization concept

- Free browsing and matching for renters.
- Small success/placement fee to seller after a successful sublease arrangement.
- Later: promoted listings, landlord/property-manager accounts, premium analytics, transaction/document services.

Future roadmap - presentation only

- **Cross-campus housing swaps:** detect opposite-direction temporary moves between universities/cities.
- **Study-abroad swaps:** identify temporary housing exchange opportunities between students moving in opposite directions.
- These are storytelling extensions only and must not consume MVP build time.

Do not prioritize for hackathon

- real payments
- legal lease contracts or e-signature
- landlord integrations
- background checks
- complex identity verification
- ML recommendation models
- production-scale infrastructure
- automatic lease transfers
- advanced maps if they delay core systems

Future expansion principle

The core matching engine should be built generically enough that a renter can search around a university, city, or work/internship location. This preserves room for internship housing and cross-city expansion without changing the MVP product promise.

20 Final Acceptance Checklist

Use this page as the completion gate before handing the MVP to judges or users.

Area	Must be true
Urgency Pricing	Shows suggested price, vacancy context, predicted match-pool change, and requires seller approval.
Date Matching	Exact-date overlap produces visible Date Match %. Date score materially affects overall match.
Renter Swiping	Renter can swipe listing cards left/right with React Native gestures and persisted decisions.
Seller Swiping	Seller can swipe renter cards left/right using the same core interaction.
Mutual Match	Two right swipes create one match and display Match Success.
Reverse Matching	Seller receives ranked compatible renters even if they have not interacted with the listing.
Partial Fill	System computes at least one real multi-renter date combination, coverage %, and estimated recovered rent.
Demand Heatmap	Seller sees renter demand by dates, price, and preferences; demand feeds pricing/recommendations.
Seller Dashboard	Displays Rent at Risk, daily vacancy loss, Rent Recovery Score, demand, pricing, match count, partial-fill.
Renter Details	Listing shows overall match, Date Match, true cost, dates, location, amenities, seller.
Mode Switching	One account can switch between renter and seller modes.
Data	Seeded dataset supports all required demo paths.

Final product definition

SubSwipe is complete only when the five core systems function together: Demand Heatmap -> Urgency Pricing -> Reverse Matching -> Date Match + Swiping -> Partial-Fill Matching.

If any of these systems is missing or only mocked with static text, the core MVP is incomplete.